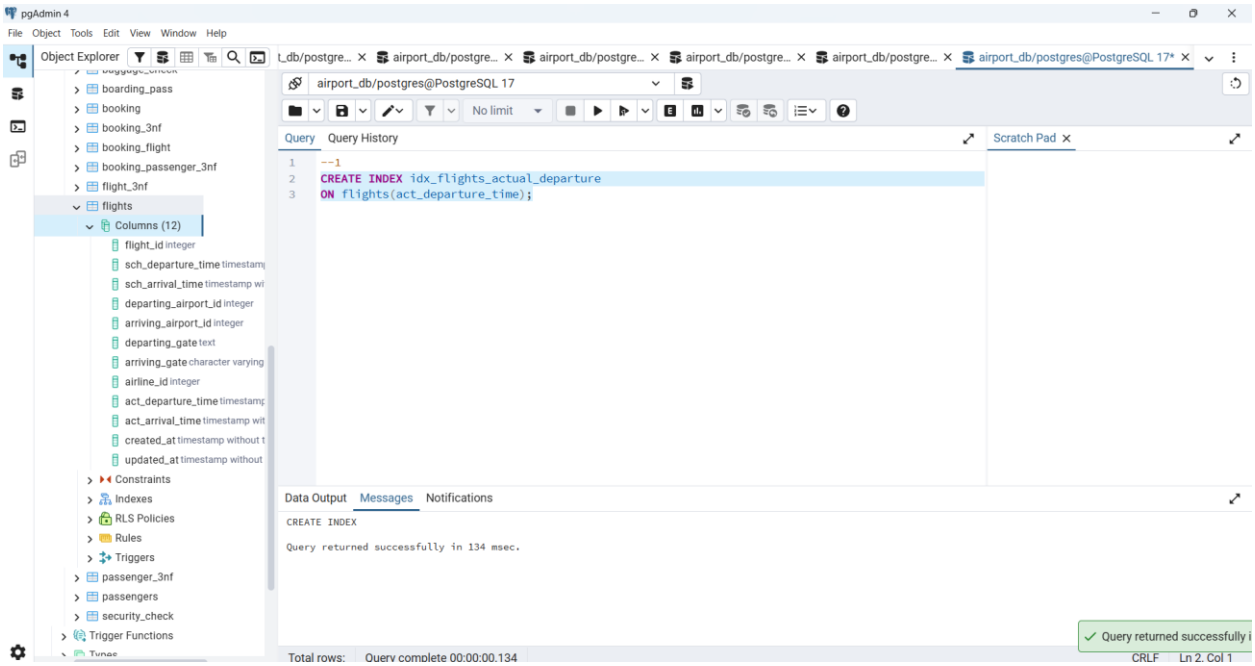
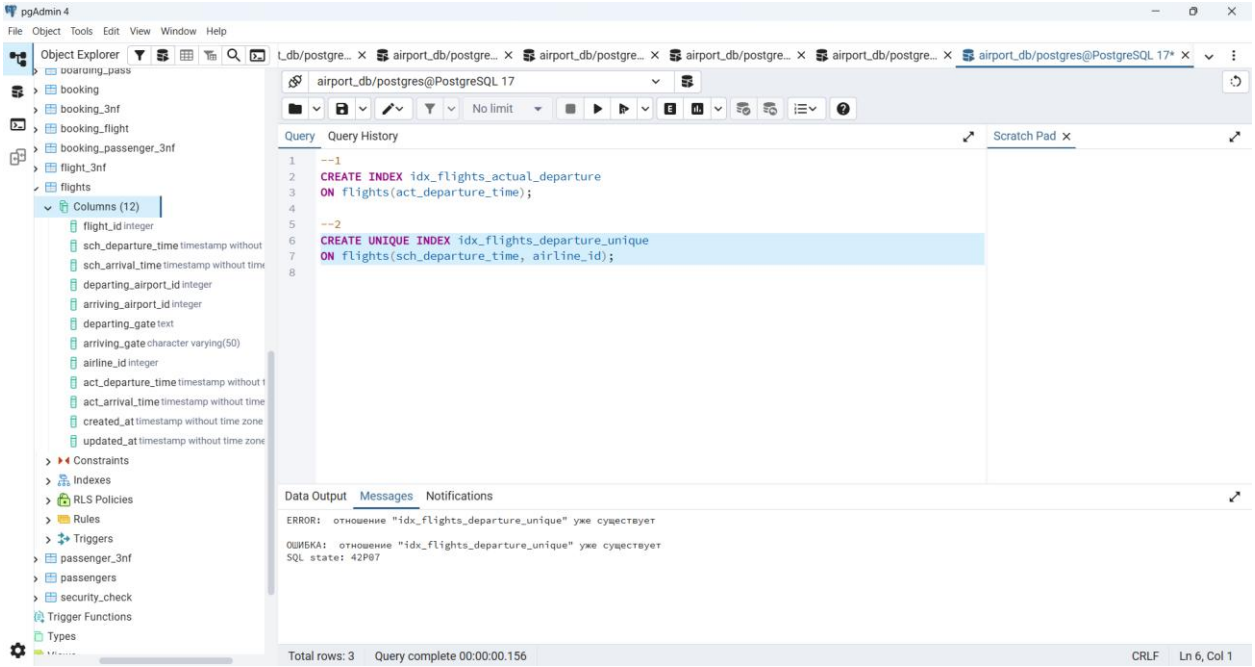
Laboratory work 7

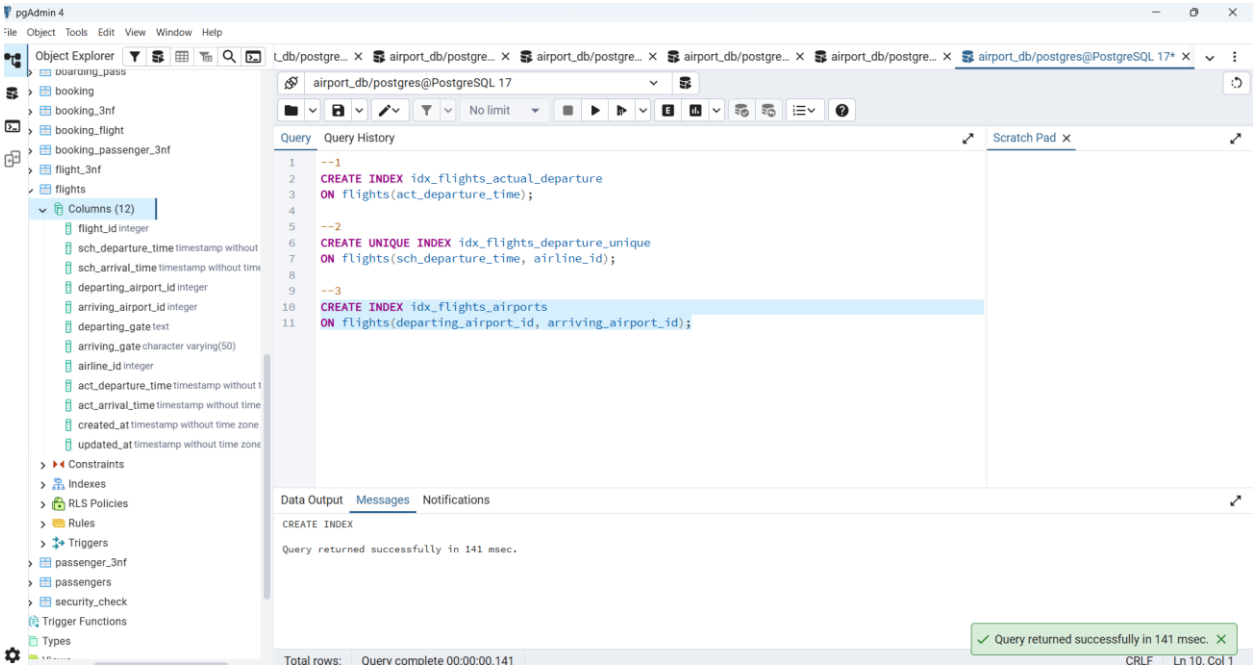
1



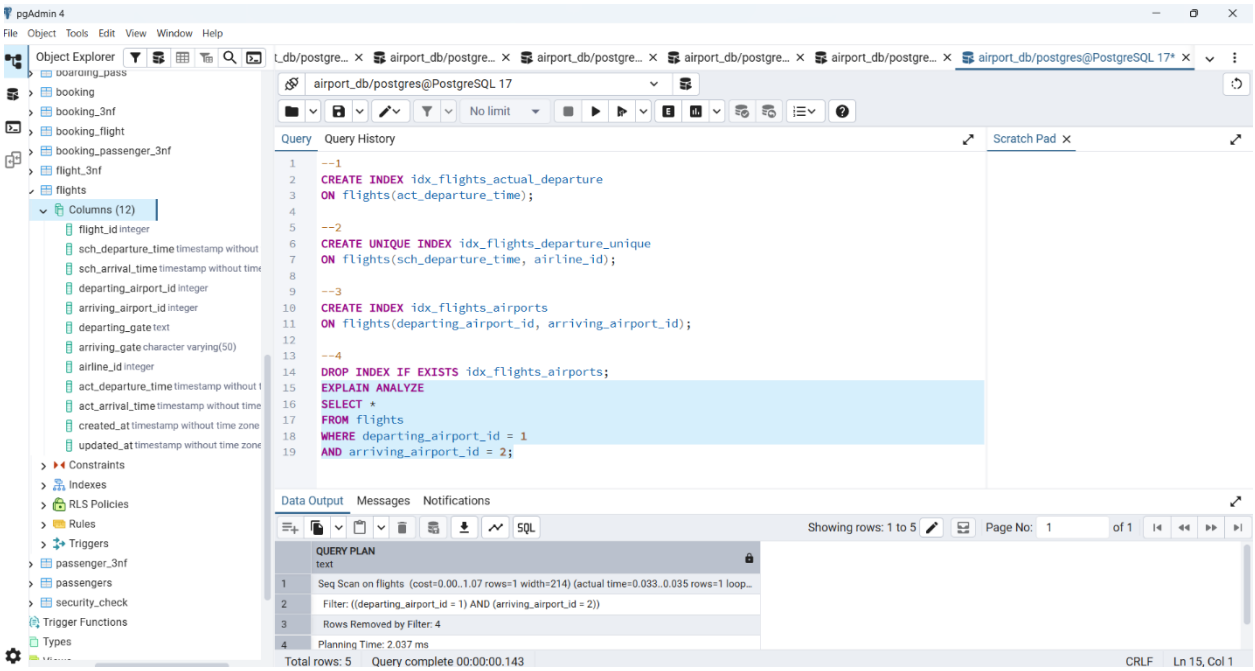
2

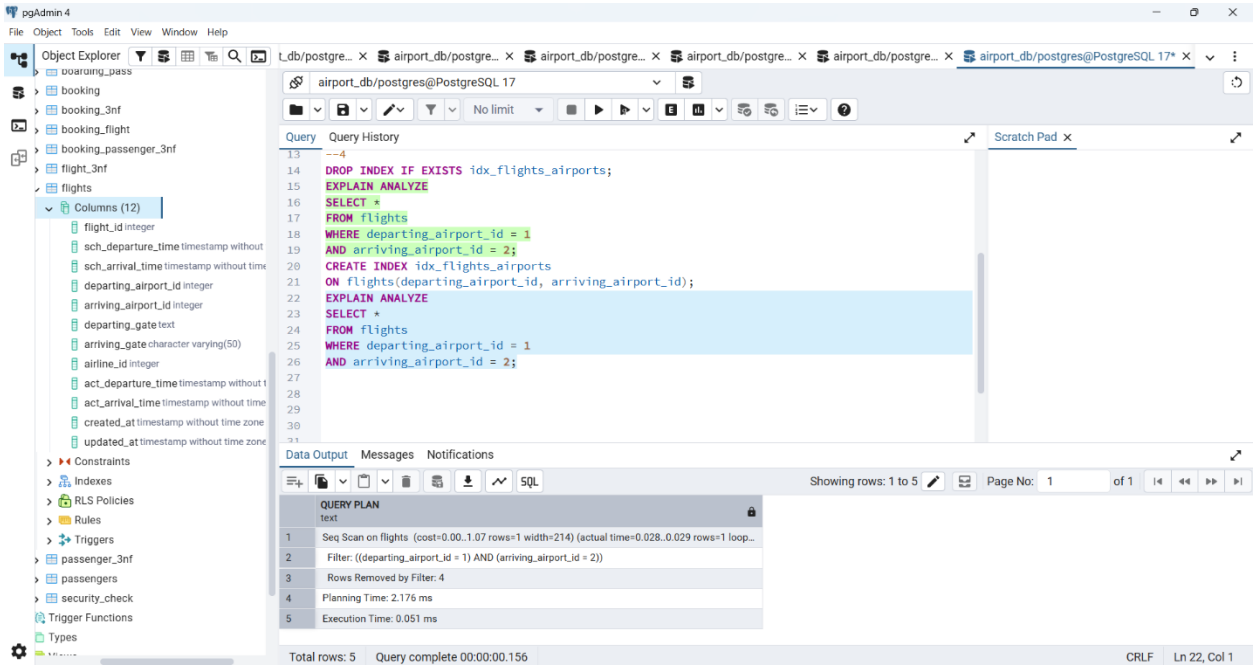


3

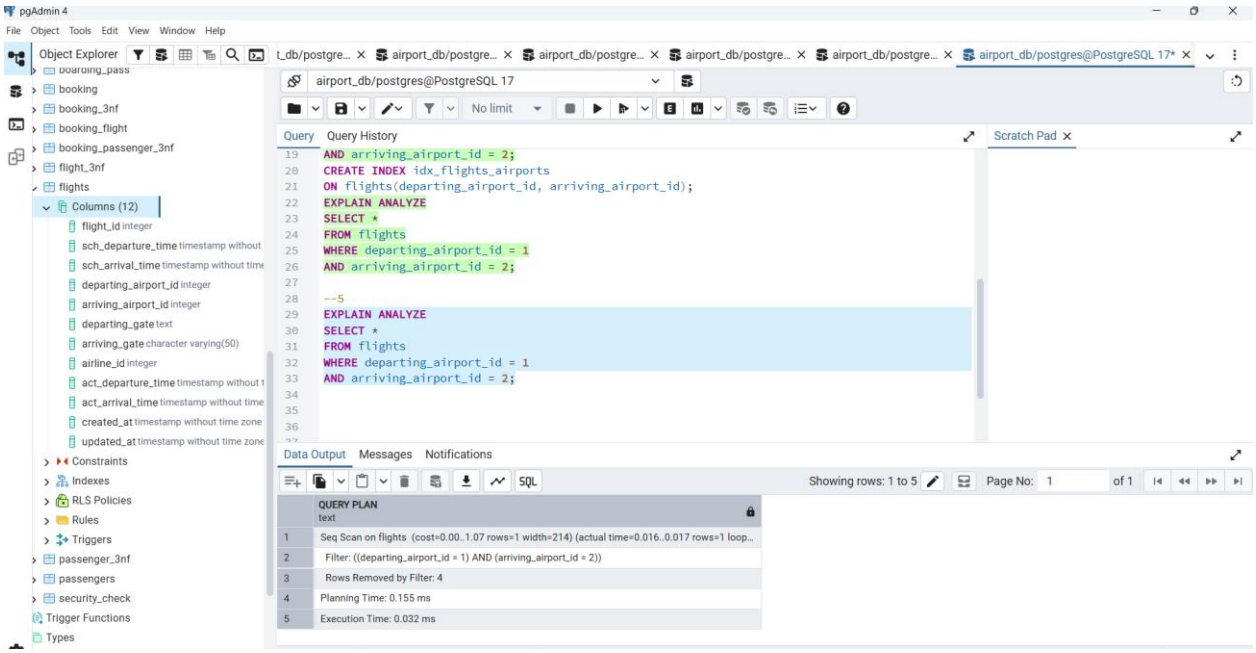

pgAdmin 4
File Object Tools Edit View Window Help
Object Explorer
airport_db/postgres@PostgreSQL 17
No limit
Query Query History
Scratch Pad
--1
CREATE INDEX idx_flights_actual_departure
ON flights(act_departure_time);
--2
CREATE UNIQUE INDEX idx_flights_departure_unique
ON flights(sch_departure_time, airline_id);
--3
CREATE INDEX idx_flights_airports
ON flights(departing_airport_id, arriving_airport_id);
Data Output Messages Notifications
CREATE INDEX
Query returned successfully in 141 msec.
Query returned successfully in 141 msec.
Total rows:
Query complete 00:00:00.141
CRLF
Ln 10, Col 1

4

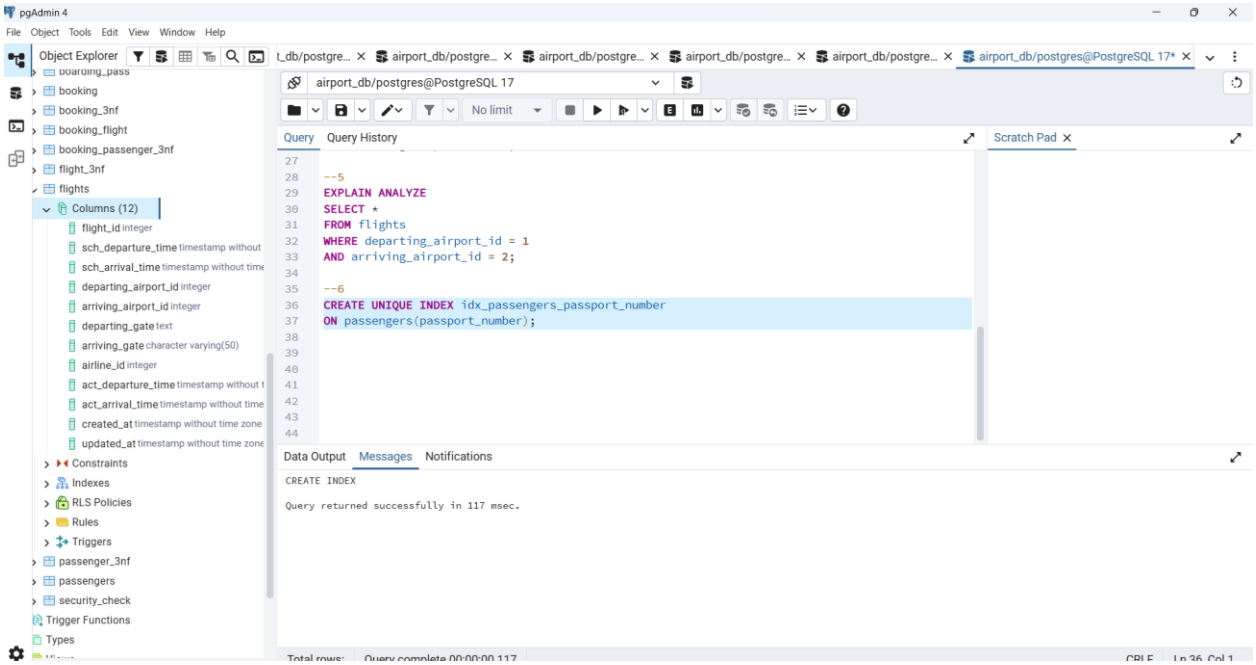

pgAdmin 4
File Object Tools Edit View Window Help
Object Explorer
airport_db/postgres@PostgreSQL 17
No limit
Query Query History
Scratch Pad
--1
CREATE INDEX idx_flights_actual_departure
ON flights(act_departure_time);
--2
CREATE UNIQUE INDEX idx_flights_departure_unique
ON flights(sch_departure_time, airline_id);
--3
CREATE INDEX idx_flights_airports
ON flights(departing_airport_id, arriving_airport_id);
--4
DROP INDEX IF EXISTS idx_flights_airports;
EXPLAIN ANALYZE
SELECT *
FROM flights
WHERE departing_airport_id = 1
AND arriving_airport_id = 2;
Data Output Messages Notifications
SQL
Showing rows: 1 to 5
Page No: 1 of 1
QUERY PLAN
text
1 Seq Scan on flights (cost=0.00..1.07 rows=1 width=214) (actual time=0.033..0.035 rows=1 loop...
2 Filter: ((departing_airport_id = 1) AND (arriving_airport_id = 2))
3 Rows Removed by Filter: 4
4 Planning Time: 2.037 ms
Total rows: 5
Query complete 00:00:00.143
CRLF
Ln 15, Col 1

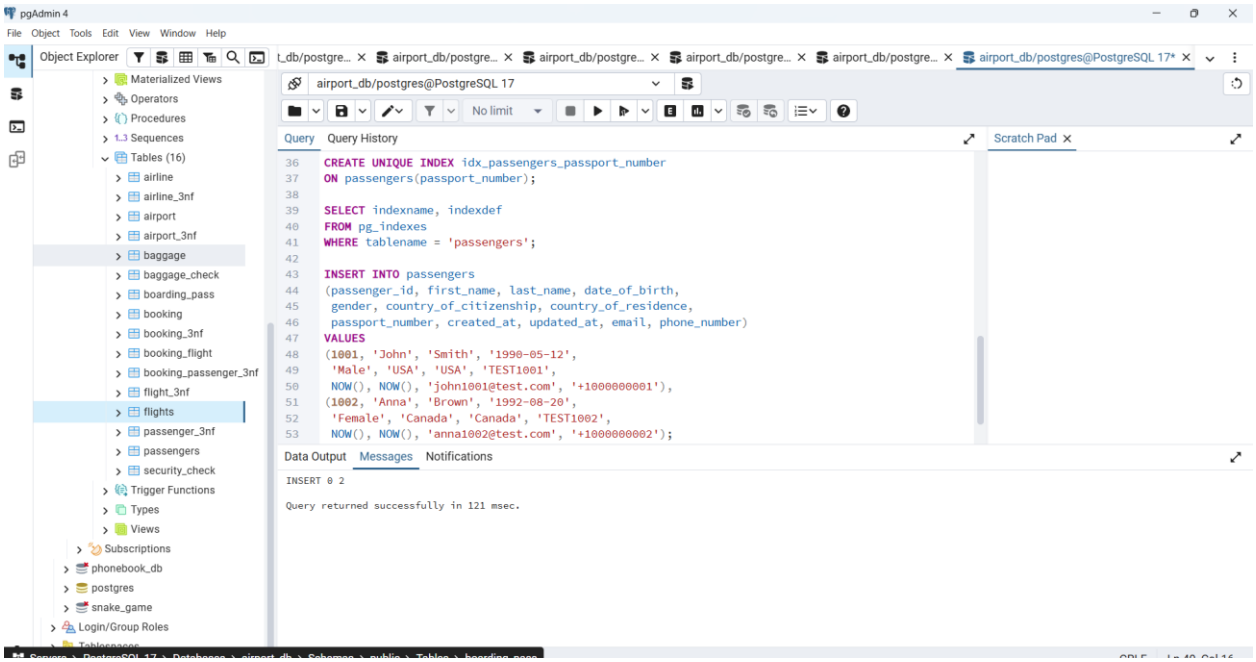

pgAdmin 4
File Object Tools Edit View Window Help
Object Explorer
airport_db/postgres@PostgreSQL 17*
airport_db/postgres@PostgreSQL 17
No limit
Query
Query History
Scratch Pad
--4
DROP INDEX IF EXISTS idx_flights_airports;
EXPLAIN ANALYZE
SELECT *
FROM flights
WHERE departing_airport_id = 1
AND arriving_airport_id = 2;
CREATE INDEX idx_flights_airports
ON flights(departing_airport_id, arriving_airport_id);
EXPLAIN ANALYZE
SELECT *
FROM flights
WHERE departing_airport_id = 1
AND arriving_airport_id = 2;
Data Output
Messages
Notifications
Showing rows: 1 to 5
Page No: 1 of 1
QUERY PLAN
text
1 Seq Scan on flights (cost=0.00..1.07 rows=1 width=214) (actual time=0.028..0.029 rows=1 loop...
2 Filter: ((departing_airport_id = 1) AND (arriving_airport_id = 2))
3 Rows Removed by Filter: 4
4 Planning Time: 2.176 ms
5 Execution Time: 0.051 ms
Total rows: 5
Query complete 00:00:00.156
CRLF
Ln 22, Col 1
booking
booking_3nf
booking_flight
booking_passenger_3nf
flight_3nf
flights
Columns (12)
flight_id integer
departing_airport_id integer
arriving_airport_id integer
departing_gate text
arriving_gate character varying(50)
airline_id integer
created_at timestamp without time zone
updated_at timestamp without time zone
Constraints
Indexes
RLS Policies
Rules
Triggers
passenger_3nf
passengers
security_check
Trigger Functions
Types

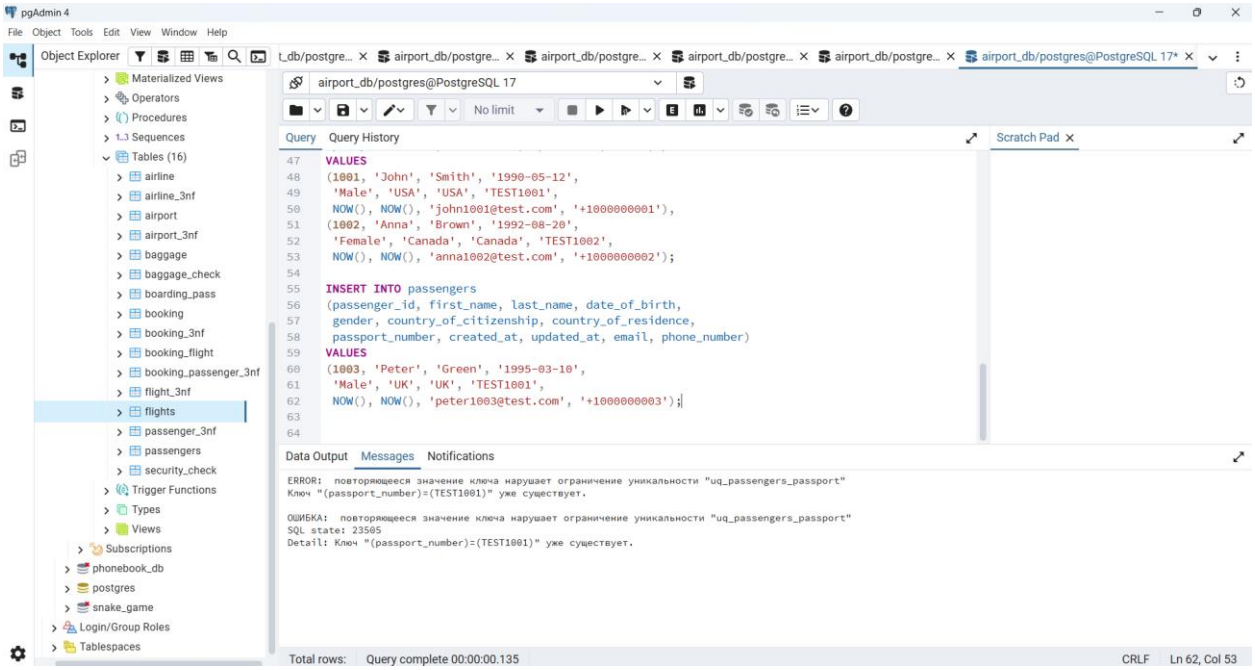
5


pgAdmin 4
File Object Tools Edit View Window Help
Object Explorer
airport_db/postgres@PostgreSQL 17*
airport_db/postgres@PostgreSQL 17
No limit
Query
Query History
Scratch Pad
AND arriving_airport_id = 2;
CREATE INDEX idx_flights_airports
ON flights(departing_airport_id, arriving_airport_id);
EXPLAIN ANALYZE
SELECT *
FROM flights
WHERE departing_airport_id = 1
AND arriving_airport_id = 2;
--5
EXPLAIN ANALYZE
SELECT *
FROM flights
WHERE departing_airport_id = 1
AND arriving_airport_id = 2;
Data Output
Messages
Notifications
Showing rows: 1 to 5
Page No: 1 of 1
QUERY PLAN
text
1 Seq Scan on flights (cost=0.00..1.07 rows=1 width=214) (actual time=0.016..0.017 rows=1 loop...
2 Filter: ((departing_airport_id = 1) AND (arriving_airport_id = 2))
3 Rows Removed by Filter: 4
4 Planning Time: 0.155 ms
5 Execution Time: 0.032 ms
booking
booking_3nf
booking_flight
booking_passenger_3nf
flight_3nf
flights
Columns (12)
flight_id integer
departing_airport_id integer
arriving_airport_id integer
departing_gate text
arriving_gate character varying(50)
airline_id integer
created_at timestamp without time zone
updated_at timestamp without time zone
Constraints
Indexes
RLS Policies
Rules
Triggers
passenger_3nf
passengers
security_check
Trigger Functions
Types

6

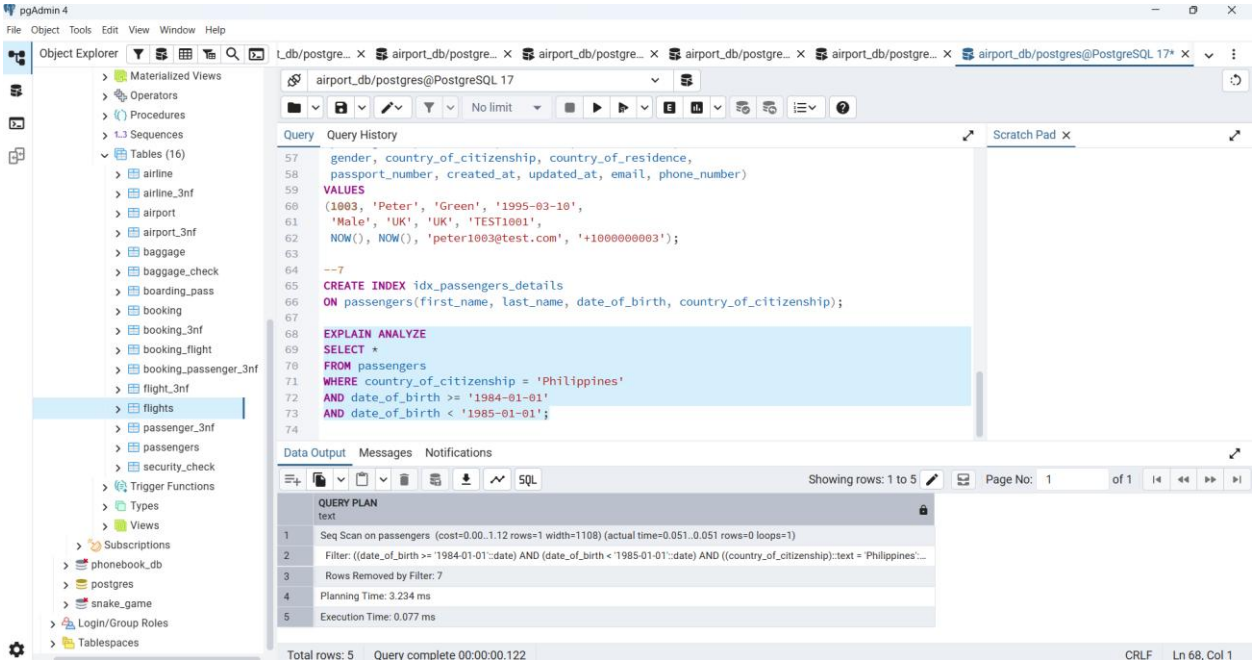

pgAdmin 4
File Object Tools Edit View Window Help
Object Explorer
airport_db/postgres@PostgreSQL 17
No limit
Query Query History
Scratch Pad
--5
EXPLAIN ANALYZE
SELECT *
FROM flights
WHERE departing_airport_id = 1
AND arriving_airport_id = 2;
--6
CREATE UNIQUE INDEX idx_passengers_passport_number
ON passengers(passport_number);
Data Output Messages Notifications
CREATE INDEX
Query returned successfully in 117 msec.


pgAdmin 4
File Object Tools Edit View Window Help
Object Explorer
airport_db/postgres@PostgreSQL 17
No limit
Query Query History
Scratch Pad
CREATE UNIQUE INDEX idx_passengers_passport_number
ON passengers(passport_number);
SELECT indexname, indexdef
FROM pg_indexes
WHERE tablename = 'passengers';
INSERT INTO passengers
(passenger_id, first_name, last_name, date_of_birth,
gender, country_of_citizenship, country_of_residence,
passport_number, created_at, updated_at, email, phone_number)
VALUES
(1001, 'John', 'Smith', '1990-05-12',
'Male', 'USA', 'USA', 'TEST1001',
NOW(), NOW(), 'john1001@test.com', '+1000000001'),
(1002, 'Anna', 'Brown', '1992-08-20',
'Female', 'Canada', 'Canada', 'TEST1002',
NOW(), NOW(), 'anna1002@test.com', '+1000000002');
Data Output Messages Notifications
INSERT 0 2
Query returned successfully in 121 msec.



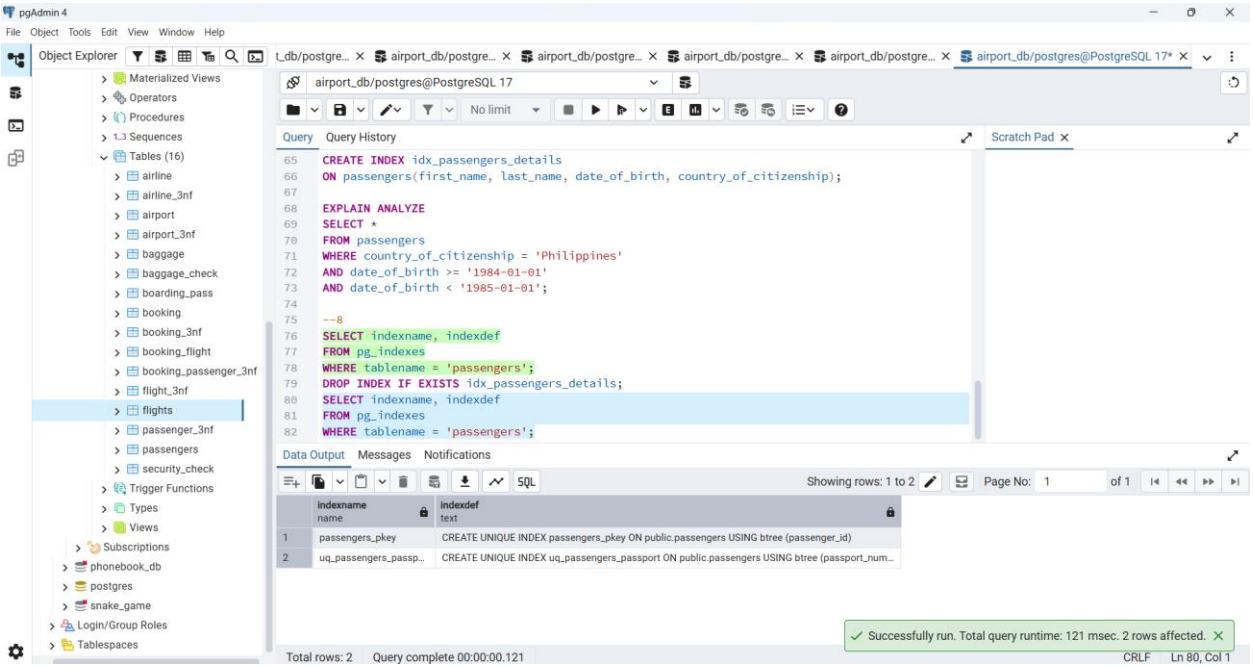
The unique index on the passport_number column was created successfully. Two new passengers were inserted into the table with different passport numbers. When I tried to insert another passenger with the same passport number (TEST1001), PostgreSQL returned error 23505. This means that the unique index prevents duplicate passport numbers and ensures data integrity

7



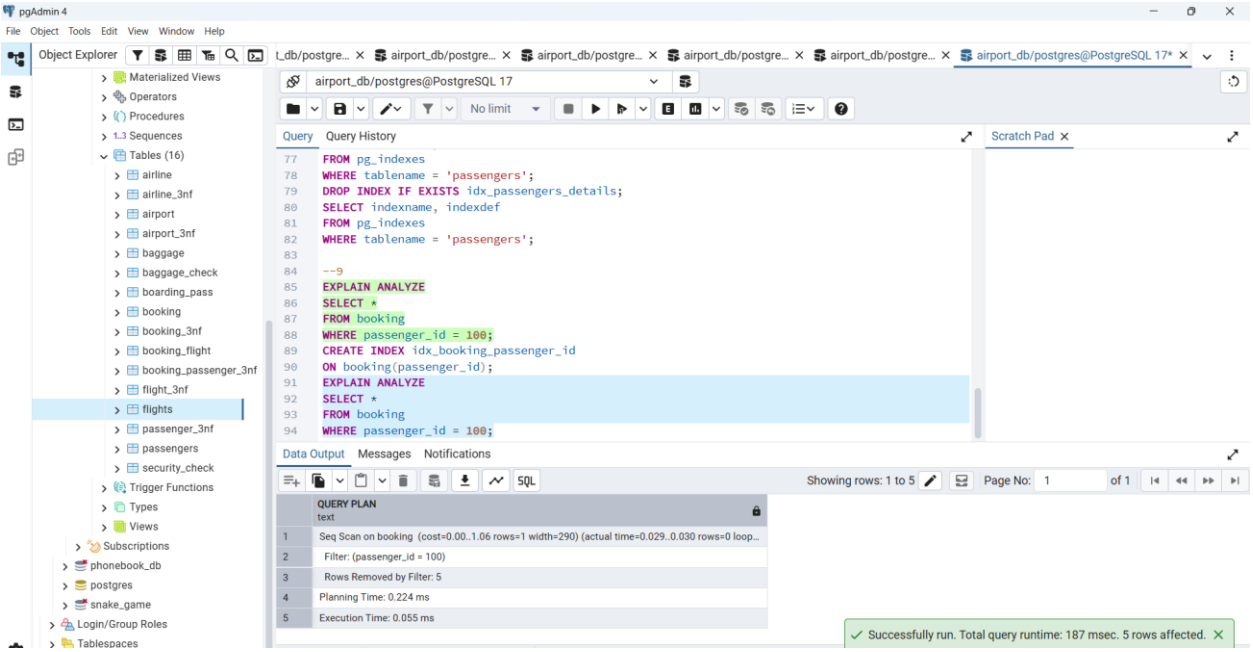
The query was executed successfully, but PostgreSQL used a Sequential Scan instead of the composite index. This happened because the query filters by country_of_citizenship and date_of_birth, while these columns are not at the beginning of the composite index. This demonstrates the leftmost prefix rule. The query returned zero rows because no passengers matched the conditions. The execution time was 0.077 ms

8



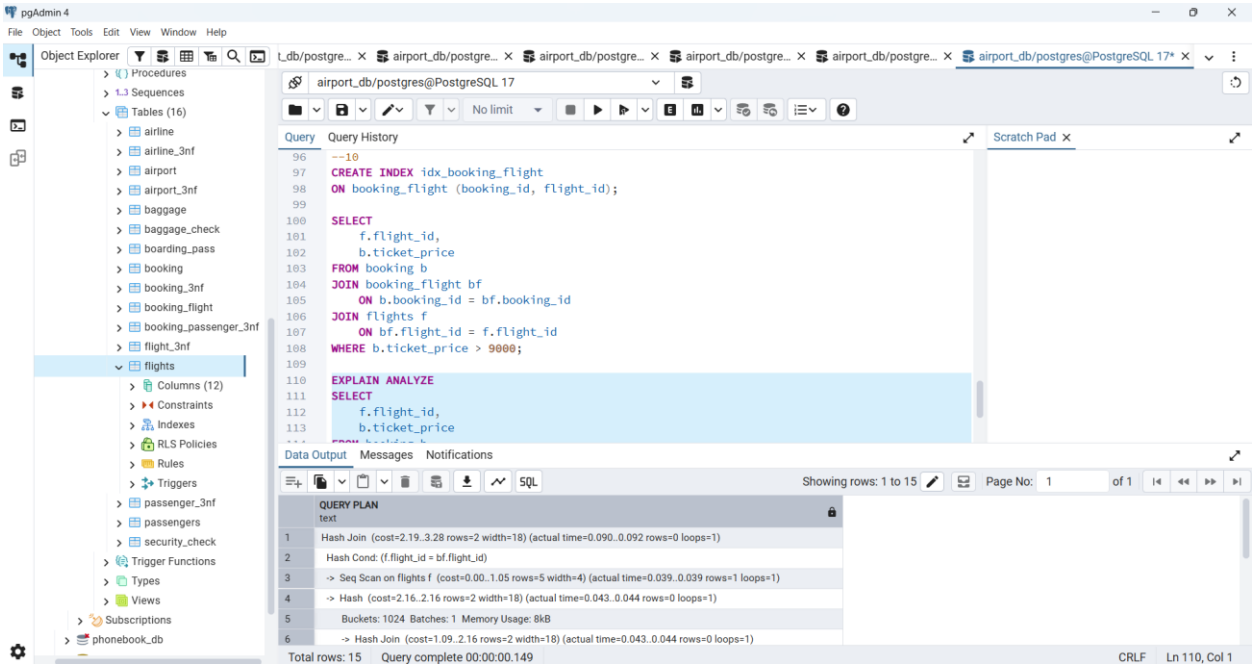
The indexes of the passengers table were successfully listed. The composite index idx_passengers_details was deleted. The primary key and unique passport number indexes remained unchanged

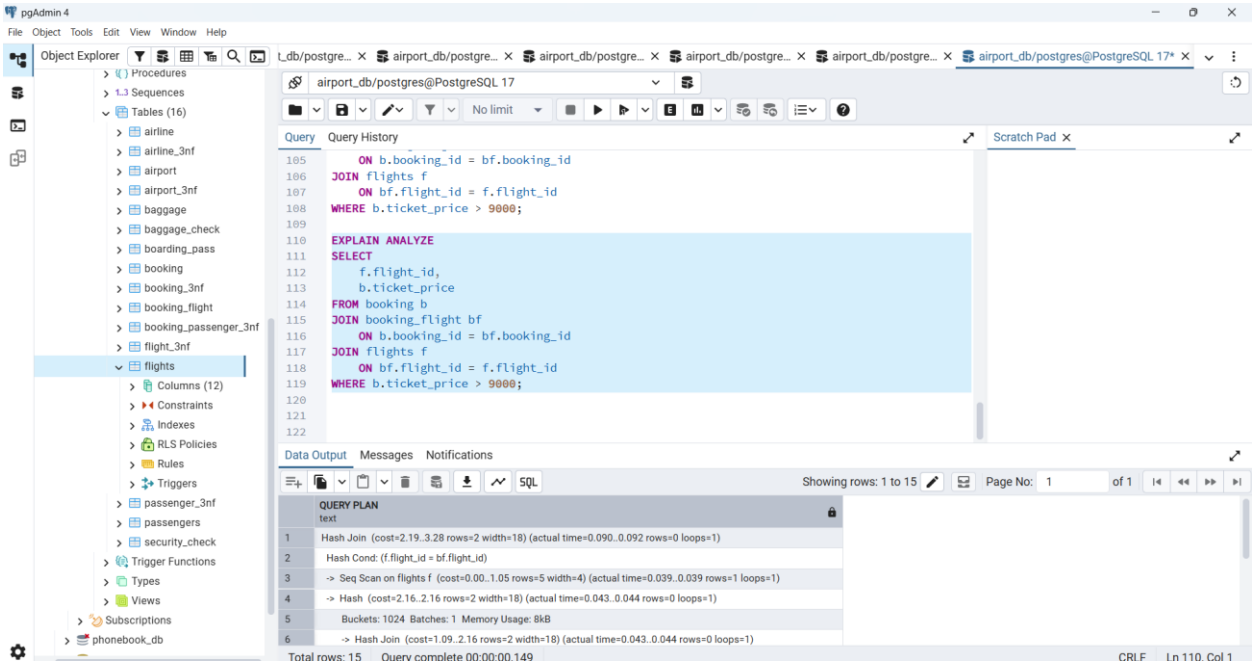
9



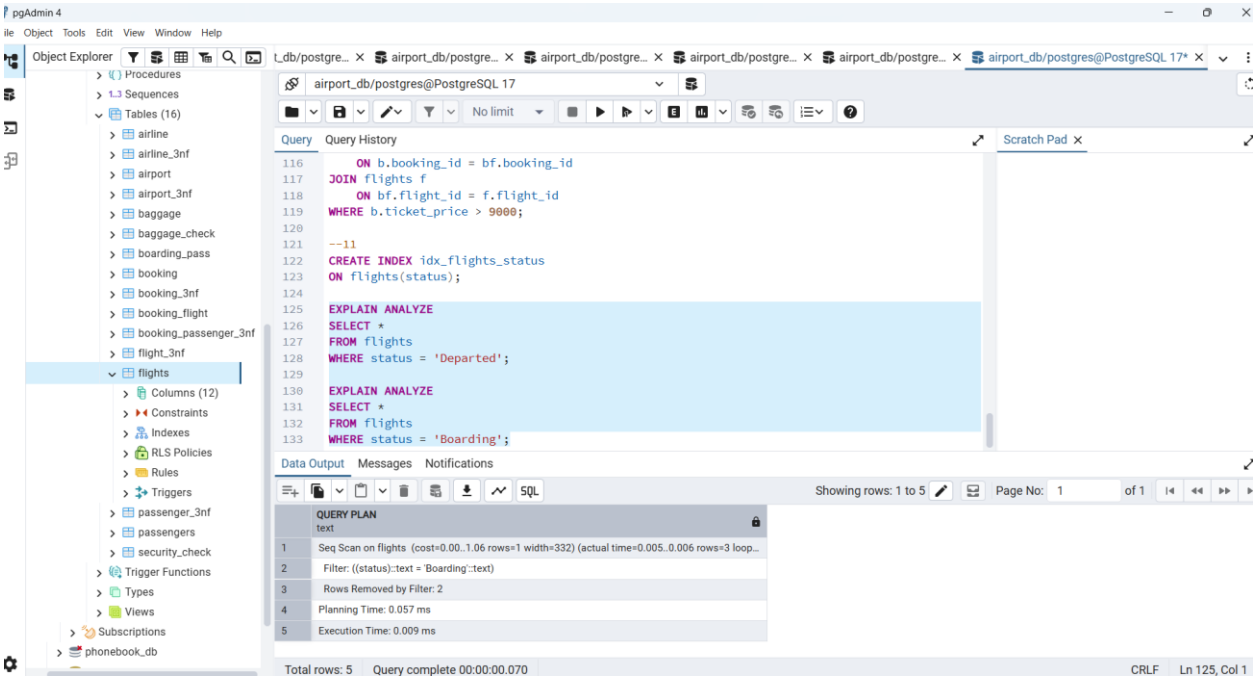
Before creating the index, the execution time was 0.066 ms. After creating the index, it was 0.056 ms. PostgreSQL continued to use Sequential Scan because the booking table contains only 5 rows. For a small table, a sequential scan can be more efficient than an index scan

10


pgAdmin 4
File Object Tools Edit View Window Help
Object Explorer
airport_db/postgres@PostgreSQL 17
Query Query History
Scratch Pad
--10
CREATE INDEX idx_booking_flight
ON booking_flight (booking_id, flight_id);
SELECT
f.flight_id,
b.ticket_price
FROM booking b
JOIN booking_flight bf
ON b.booking_id = bf.booking_id
JOIN flights f
ON bf.flight_id = f.flight_id
WHERE b.ticket_price > 9000;
EXPLAIN ANALYZE
SELECT
f.flight_id,
b.ticket_price
Data Output Messages Notifications
Showing rows: 1 to 15
Page No: 1 of 1
QUERY PLAN
text
Hash Join (cost=2.19..3.28 rows=2 width=18) (actual time=0.090..0.092 rows=0 loops=1)
Hash Cond: (f.flight_id = bf.flight_id)
-> Seq Scan on flights f (cost=0.00..1.05 rows=5 width=4) (actual time=0.039..0.039 rows=1 loops=1)
-> Hash (cost=2.16..2.16 rows=2 width=18) (actual time=0.043..0.044 rows=0 loops=1)
Buckets: 1024 Batches: 1 Memory Usage: 8kB
-> Hash Join (cost=1.09..2.16 rows=2 width=18) (actual time=0.043..0.044 rows=0 loops=1)
Total rows: 15 Query complete 00:00:00.149
CRLF Ln 110, Col 1


pgAdmin 4
File Object Tools Edit View Window Help
Object Explorer
airport_db/postgres@PostgreSQL 17
Query Query History
Scratch Pad
ON b.booking_id = bf.booking_id
JOIN flights f
ON bf.flight_id = f.flight_id
WHERE b.ticket_price > 9000;
EXPLAIN ANALYZE
SELECT
f.flight_id,
b.ticket_price
FROM booking b
JOIN booking_flight bf
ON b.booking_id = bf.booking_id
JOIN flights f
ON bf.flight_id = f.flight_id
WHERE b.ticket_price > 9000;
Data Output Messages Notifications
Showing rows: 1 to 15
Page No: 1 of 1
QUERY PLAN
text
Hash Join (cost=2.19..3.28 rows=2 width=18) (actual time=0.090..0.092 rows=0 loops=1)
Hash Cond: (f.flight_id = bf.flight_id)
-> Seq Scan on flights f (cost=0.00..1.05 rows=5 width=4) (actual time=0.039..0.039 rows=1 loops=1)
-> Hash (cost=2.16..2.16 rows=2 width=18) (actual time=0.043..0.044 rows=0 loops=1)
Buckets: 1024 Batches: 1 Memory Usage: 8kB
-> Hash Join (cost=1.09..2.16 rows=2 width=18) (actual time=0.043..0.044 rows=0 loops=1)
Total rows: 15 Query complete 00:00:00.149
CRLF Ln 110, Col 1

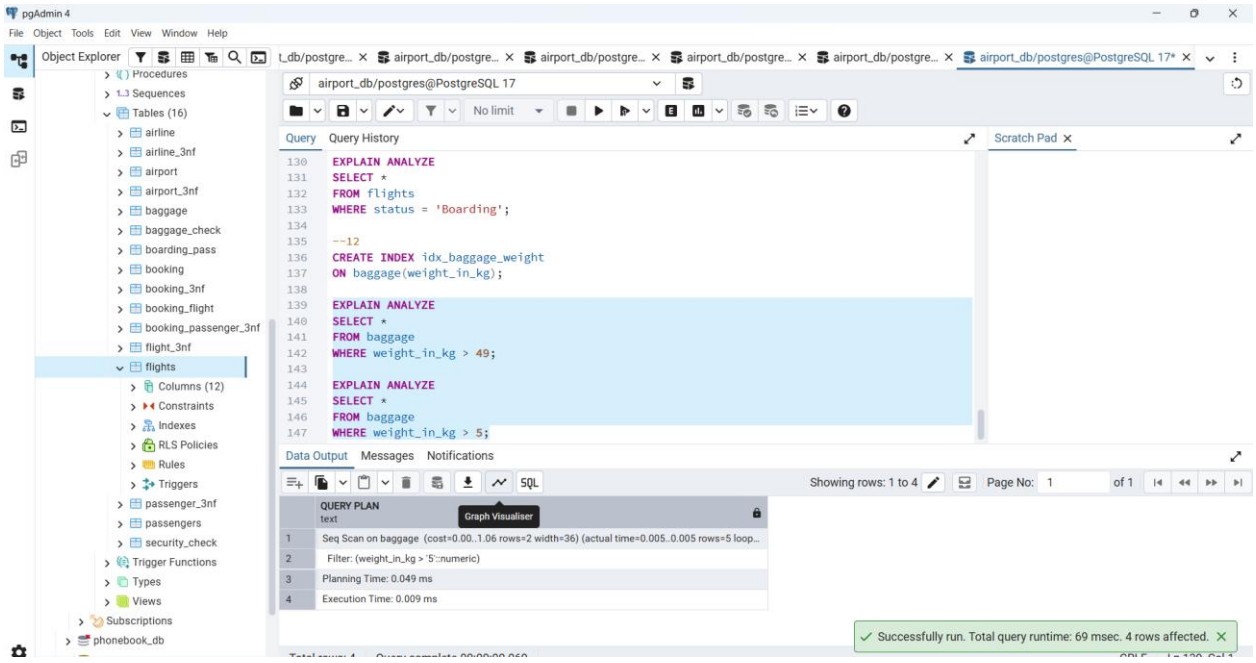
11



The optimizer uses the index for status = 'Departed' if it returns fewer rows. For status = 'Boarding', it may ignore the index if many rows match and use a Sequential Scan instead.

This happens because of selectivity. When a query returns most of the table's rows, a Sequential Scan is often faster than an Index Scan

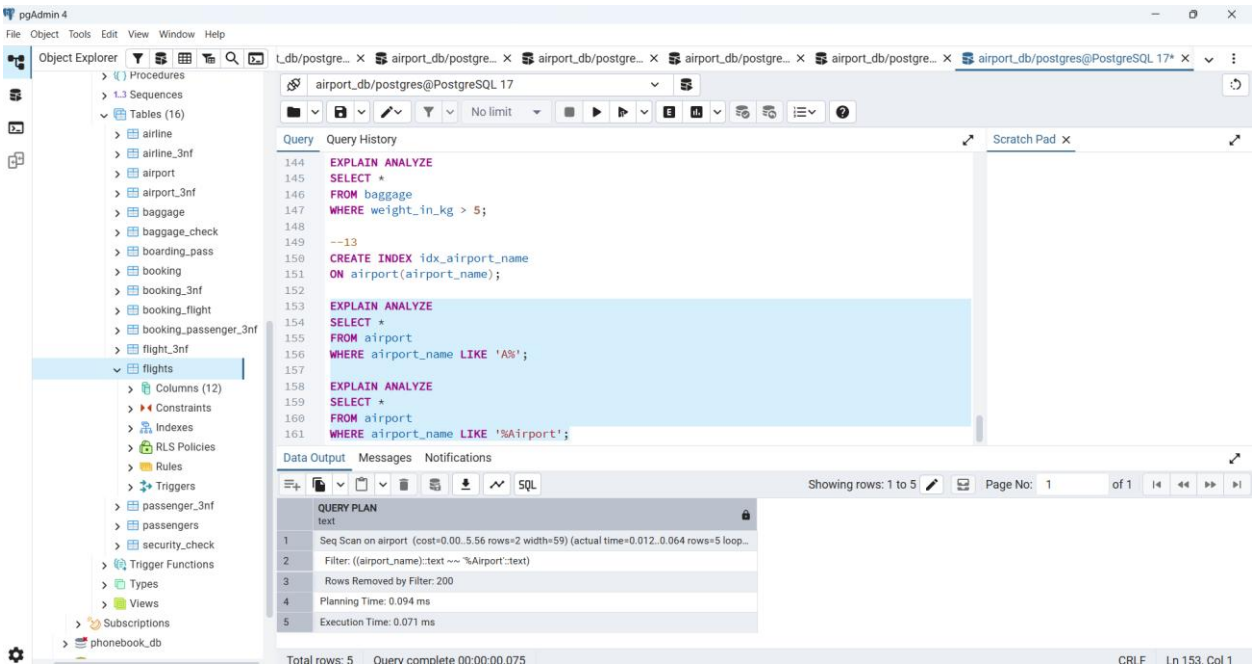
12



weight_in_kg > 49: Returns a small number of rows. PostgreSQL may use an Index Scan because the index helps find matching rows quickly.

weight_in_kg > 5: Returns almost all rows. PostgreSQL may use a Sequential Scan because reading the whole table is more efficient

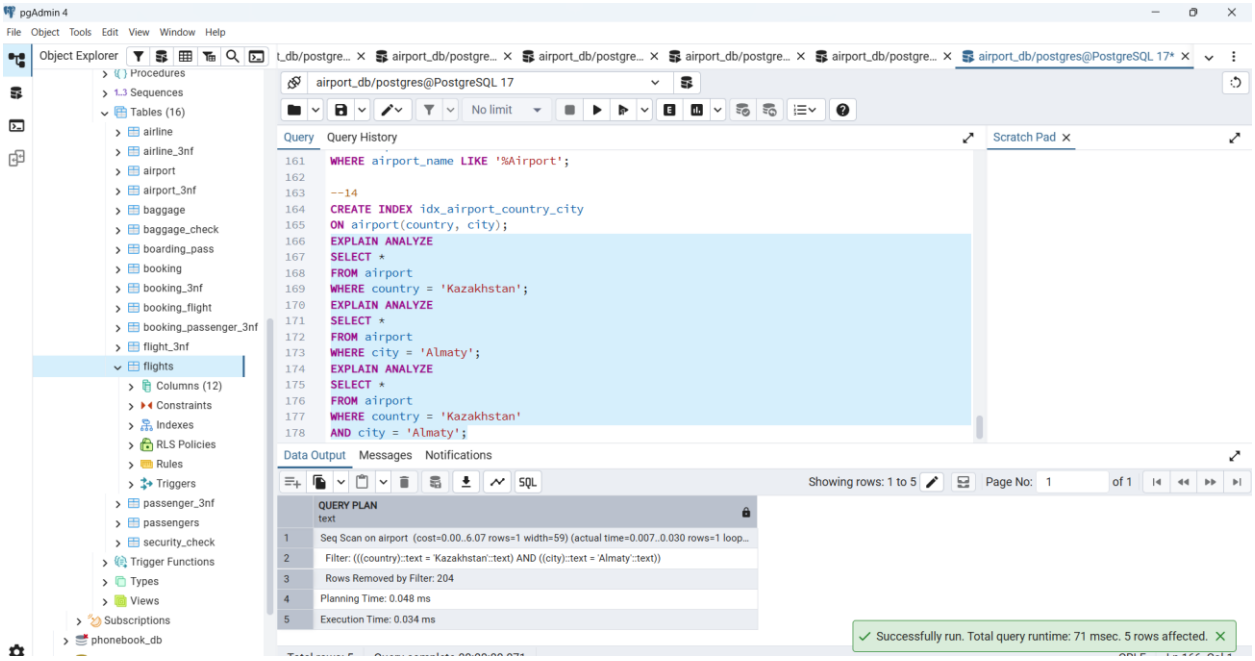
13



LIKE 'A%' — The index may be used because the search pattern starts with the letter A.

LIKE '%Airport' — A regular B-tree index generally cannot be used efficiently because the search pattern starts with a wildcard and ends with the word Airport

14

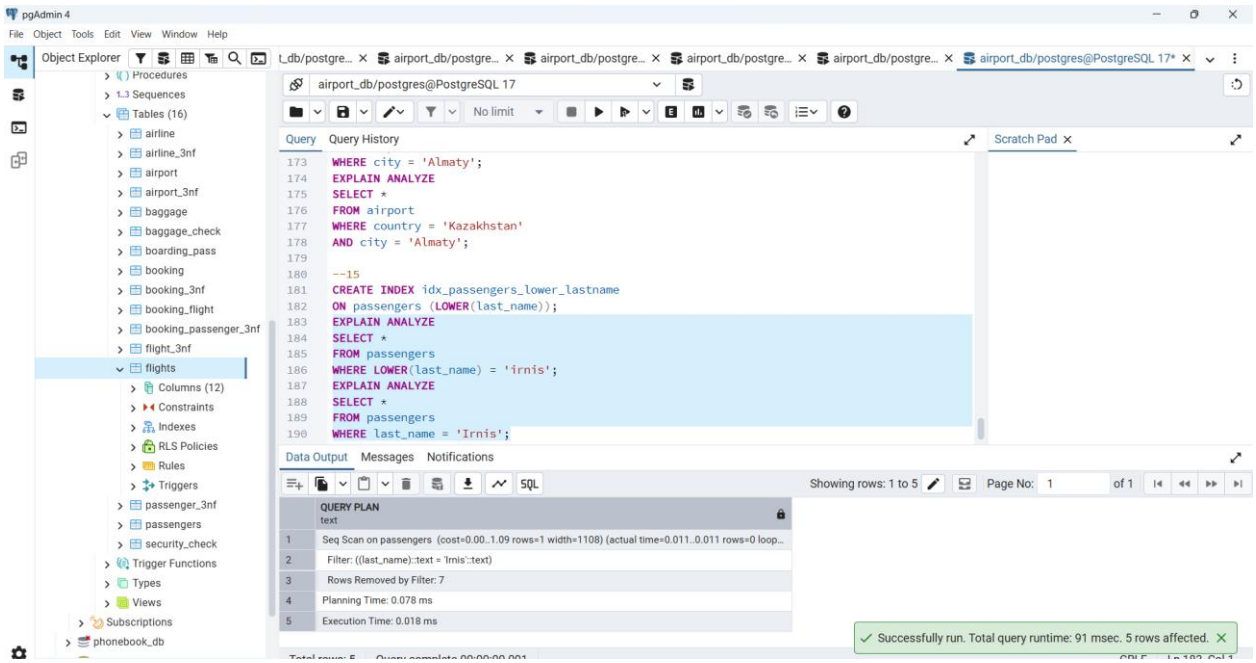


The composite index contains two columns: country and city.

The leftmost prefix rule means that the index can be used when filtering by the first column (country) or by both columns (country and city)

However, filtering by city alone generally cannot use this index efficiently because city is the second column

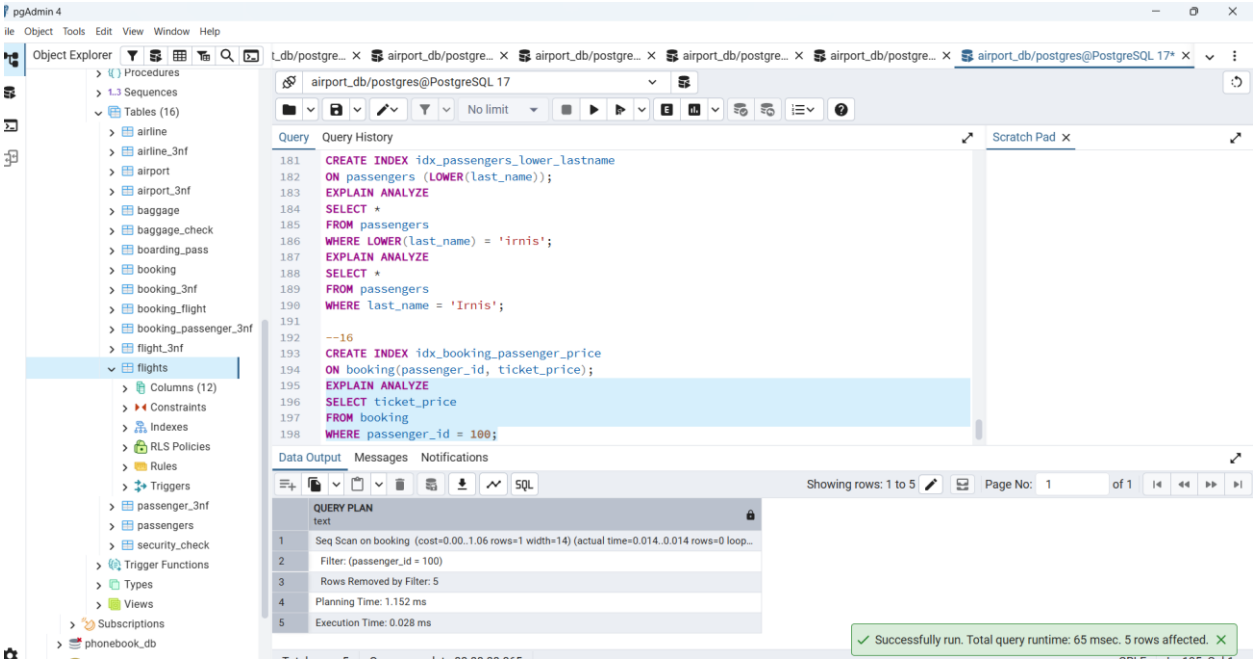
15



WHERE LOWER(last_name) = 'irnis' — This query can use the functional index because the query expression matches the indexed expression LOWER(last_name).

WHERE last_name = 'Irnis' — This query generally cannot use the functional index because it searches the original column instead of the lowercase expression

16

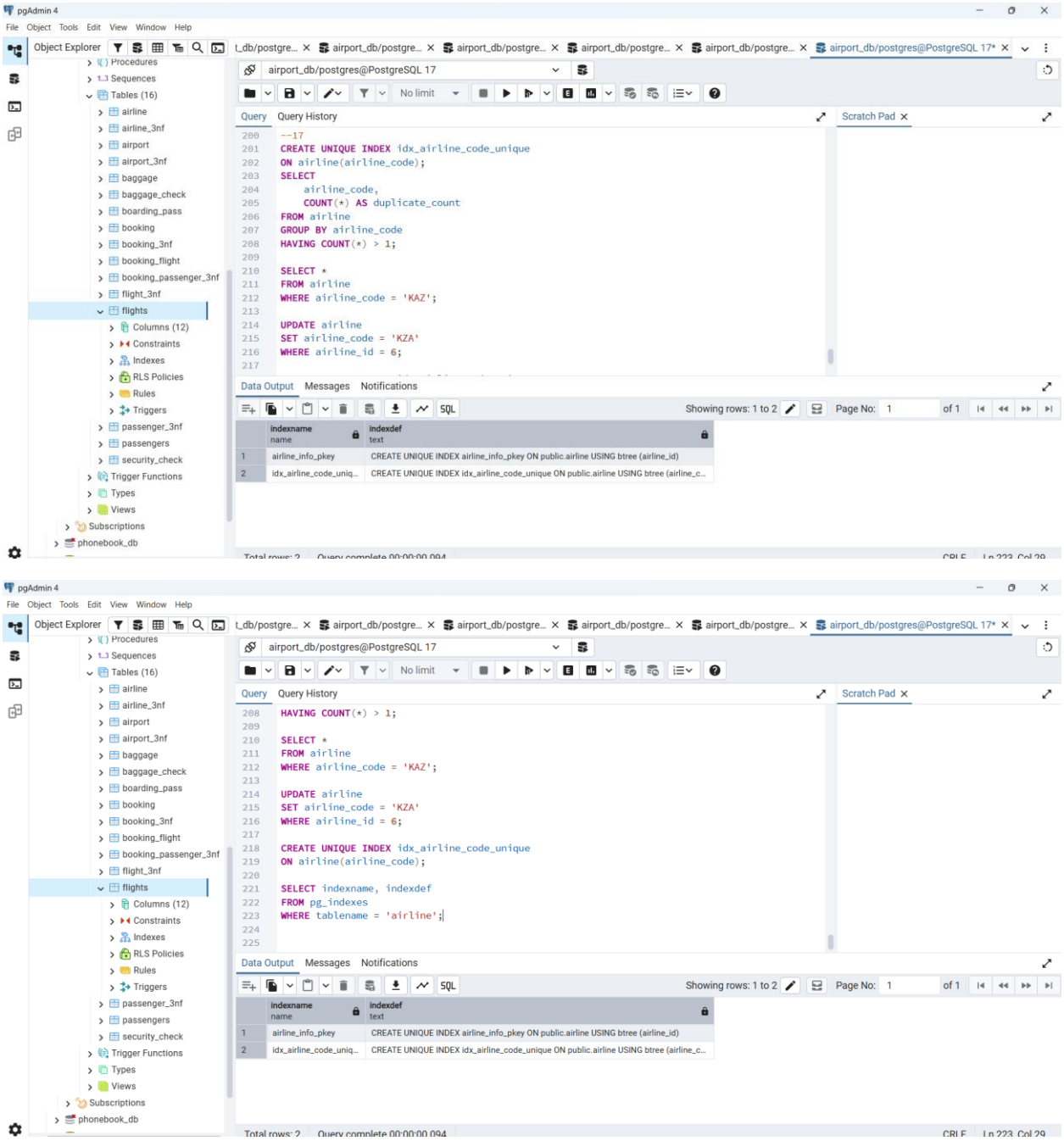


A covering index contains all the columns required by a query

In this example, the index contains passenger_id and ticket_price. The query uses passenger_id to find records and retrieves only ticket_price.

Therefore, PostgreSQL can use an Index Only Scan without reading the actual table rows, provided the visibility map allows it.

17



The error occurred because the value KAZ appeared twice in the airline_code column. PostgreSQL does not allow duplicate values when creating a unique index

I found the duplicate records using GROUP BY and HAVING COUNT(*) > 1. Then I corrected one duplicate value and successfully created the unique index